

Resumo de Java — Dataprev 2026

Prova FGV em 11/10/2026 · 70 questões · foco em pegadinhas de código

1. JVM e plataforma

- O código `.java` é compilado pelo `javac` para **bytecode** (`.class`), que a JVM executa ("escreva uma vez, rode em qualquer lugar").
- **JDK** contém a **JRE**, que contém a **JVM**. JDK = compilador + ferramentas; JRE = JVM + bibliotecas.
- **Heap**: objetos. **Stack**: variáveis locais e chamadas de método. O **Garbage Collector** libera objetos sem referência.

2. Sintaxe

- Java diferencia maiúsculas de minúsculas (case sensitive).
- Ponto de entrada: `public static void main(String[] args)`.
- Classe pública deve ter o mesmo nome do arquivo. Convenção: classes em CamelCase, métodos e variáveis em camelCase.

3. Variáveis e tipos primitivos

- **8 primitivos**: byte (8 bits), short (16), int (32), long (64), float (32), double (64), char (16, Unicode), boolean.
- **Atributos** têm valor padrão (0, 0.0, false, null). **Variáveis locais** não têm: usar sem inicializar dá erro de compilação.

4. Literais e casting

- Long termina em L, float em f. Prefixos: 0x hexadecimal, 0b binário, 0 octal.
- Conversão para tipo maior é implícita (widening); para tipo menor exige cast explícito (narrowing).
- `int / int` é divisão inteira. `byte + byte` resulta em `int`.

5. Operadores

- `++/- -`: pré-incremento usa o valor depois, pós-incremento usa o valor antes.
- `&&` e `||` têm curto-circuito; `&` e `|` não.
- Em objetos, `==` compara **referência**; `equals()` compara **conteúdo**.
- Atenção à precedência e ao + concatenando String (ex.: `1 + 2 + "a"` dá `"3a"`).

6. Controle de fluxo

- switch sem break cai nos casos seguintes (**fall-through**). Aceita byte, short, char, int, wrappers, String e enum.
- do-while executa ao menos uma vez. break sai do laço; continue pula para a próxima iteração.

7. Arrays

- Tamanho fixo, índice começa em 0. `length` é **atributo** (sem parênteses), diferente de `String.length()`.
- Acesso fora do limite lança `ArrayIndexOutOfBoundsException`. São objetos e vão por referência para métodos.

8. Classes, atributos e métodos

- `static` pertence à classe, não ao objeto.
- **Sobrecarga**: mesmo nome, parâmetros diferentes (só o retorno diferente não vale).
- Acesso: `private` < padrão (pacote) < `protected` < `public`.
- Java sempre passa por valor (em objetos, passa o valor da referência).

9. Construtores

- Mesmo nome da classe, sem tipo de retorno. O construtor padrão só existe se nenhum for declarado.
- `this()` chama outro construtor da classe; `super()` chama o da superclasse e deve ser a **primeira linha**.

10. Herança

- `extends`, apenas herança simples entre classes. Construtores não são herdados. Toda classe herda de `Object`.
- `final` impede herdar (classe) ou sobrescrever (método).
- **Sobrescrita**: mesma assinatura, visibilidade igual ou maior, use `@Override`.

11. Polimorfismo

- `Pai p = new Filho()`: métodos de instância são resolvidos em **tempo de execução** (pelo objeto real).
- Atributos e métodos `static` são resolvidos pelo **tipo da referência**, não pelo objeto.
- Sobrecarga é decidida na compilação; sobrescrita, na execução. Cast incorreto lança `ClassCastException`.

12. Classes abstratas

- Não podem ser instanciadas. Podem ter construtor, atributos, métodos concretos e abstratos.
- Subclasse não abstrata deve implementar todos os métodos abstratos.

13. Interfaces

- Métodos são `public abstract` por padrão; atributos são `public static final`.
- Uma classe implementa várias interfaces. Desde o Java 8: métodos `default` e `static`.
- **Abstrata x interface**: abstrata tem estado e construtor; interface permite herança múltipla.

14. Tratamento de exceções

- Hierarquia: `Throwable` > `Error` e `Exception` > `RuntimeException`.
- **Checked**: o compilador obriga tratar ou declarar (`throws`). **Unchecked**: `RuntimeException` e filhas.
- `finally` roda quase sempre (exceção: `System.exit()`).
- Vários `catch`: o mais **específico primeiro**, senão erro de compilação.
- `throw` lança; `throws` declara. `try-with-resources` fecha recursos `AutoCloseable` sozinho.

15. Tipos enumerados (enum)

- Constantes implicitamente `public static final`. Métodos: `values()`, `valueOf()`, `ordinal()`, `name()`.
- Podem ter atributos, construtor (privado) e métodos; servem em `switch`. Não estendem classes, mas podem implementar interfaces.

16. Classes wrappers

- `Integer`, `Double` etc. **Autoboxing/unboxing** automáticos. Unboxing de `null` lança `NullPointerException`.
- **Pegadinha**: `Integer` de -128 a 127 é cacheado, então `==` dá `true` nessa faixa e `false` fora dela. Use `equals`.

17. Strings

- **Imutáveis**; literais ficam no **string pool**.
- `"a" == "a"` é true; `new String("a") == "a"` é false. Para conteúdo, use `equals`.
- `StringBuilder`: mutável, não sincronizado. `StringBuffer`: mutável, sincronizado.
- Métodos comuns: `substring`, `indexOf`, `charAt`, `equalsIgnoreCase`, `split`, `replace`, `trim`.

18. Coleções (muito cobrado)

- **List** (ordem, aceita duplicatas): `ArrayList`, `LinkedList`.
- **Set** (sem duplicatas): `HashSet` (sem ordem), `LinkedHashSet` (ordem de inserção), `TreeSet` (ordenado).
- **Map** (não estende `Collection`): `HashMap` (aceita uma chave null), `TreeMap` (ordenado, sem chave null), `Hashtable` (sincronizado, sem null).
- Se sobrescrever `equals`, sobrescreva `hashCode` também.
- `Comparable` (`compareTo`, ordem natural) x `Comparator` (`compare`, ordem externa).
- Remover elemento durante `for-each` lança `ConcurrentModificationException`.
- Generics: `List<Object>` **não é** `List<String>`.

19. Threads

- Criar com `Thread` ou `Runnable`. `start()` cria a thread; chamar `run()` direto **não cria**.
- Estados: `NEW`, `RUNNABLE`, `BLOCKED`, `WAITING`, `TIMED_WAITING`, `TERMINATED`.
- `synchronized` evita condição de corrida. `sleep()` não libera o lock; `wait()` libera. `join()` espera outra thread terminar.

Prioridade se faltar tempo: orientação a objetos (herança, polimorfismo, abstratas e interfaces), exceções, coleções e Strings. São os temas que mais rendem questões do tipo "qual a saída?".